Agentic Appeals — Phase 1 MVP

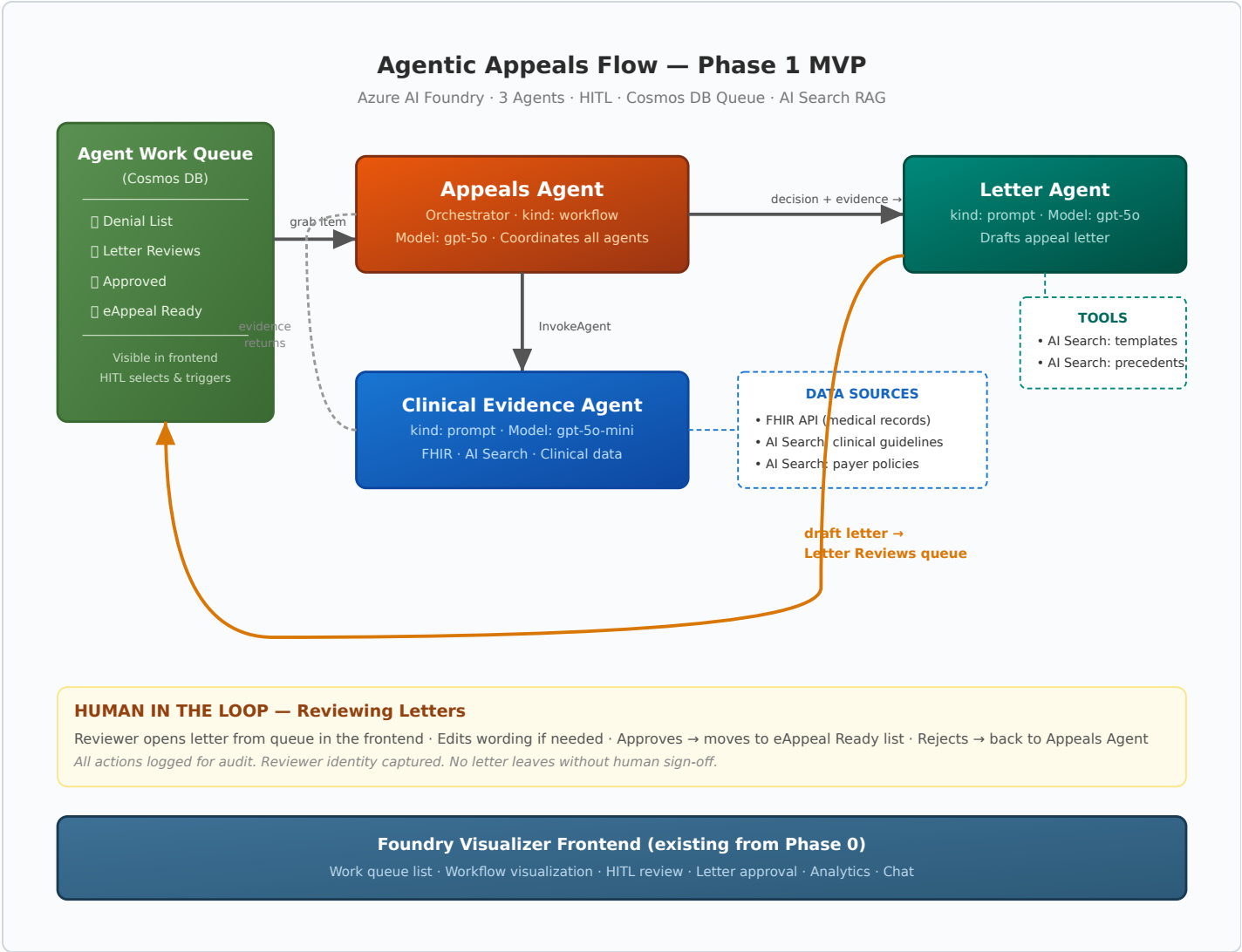
Infrastructure Resource Sheet – April 9, 2026 *(manual provisioning — quick setup for stakeholder demo)*

Objective

Get the base-level Azure architecture provisioned so the engineering team can start wiring up the Agentic Appeals flow and show results to stakeholders as quickly as possible. Infra provisions resources manually in the portal now; Terraform and pipelines come later. The goal is a **working MVP in the fastest possible time** — not production hardening.

> **Quick-win mindset:** Every resource below has a "why now" justification. If it's on this list, the engineering team is blocked without it. Anything that can wait is explicitly called out as "later." Provision top-to-bottom — the order matters because of dependencies.

Architecture Overview



Resource Spec Sheet — Provision In This Order

Infra team provisions these manually in the Azure Portal. Order matters — later resources reference earlier ones. All resources go into the **existing MLAI Resource Group**.

#	Resource	Name	SKU	Setup Notes
AI FOUNDRY — Core AI Platform				
1	**Azure AI Foundry Account**	`foundry-appeals`	S0	Kind: `AIServices`. Enable system-assigned managed identity. Set custom domain to `foundry-appeals`. Same region as other MLAI resources.
2	**Foundry Project**	`appeals-project`	—	Create inside the Foundry account via the AI Foundry portal. This is the workspace where agents, connections, and deployments live.
3	**Model: gpt-5o**	`gpt-5o`	Standard	Deploy inside the Foundry account. Used by **Appeals Agent** (orchestrator) and **Letter Agent**. Set TPM quota to **80K** to start. These agents need strong reasoning and writing.
4	**Model: gpt-5o-mini**	`gpt-5o-mini`	Standard	Deploy inside the Foundry account. Used by **Clinical Evidence Agent**. Set TPM quota to **80K**. This agent does retrieval-heavy work — mini is cost-effective and fast enough.
5	**Model: text-embedding-3-large**	`text-embedding-3-large`	Standard	Deploy inside the Foundry account. Used by AI Search for vector embeddings in the RAG indexes. Set TPM quota to **350K**. This runs during index build and every search query.
AI FOUNDRY — Agent Definitions (created by engineering in the Foundry portal after infra provisions the account)				
6	**Appeals Agent**	`appeals-agent`	workflow	**Orchestrator.** Uses gpt-5o. Grabs denial from work queue, invokes Clinical Evidence Agent, makes approve/deny decision, invokes Letter Agent, routes draft to HITL review queue. Workflow YAML with `InvokeAzureAgent` + `Question` actions.
7	**Clinical Evidence Agent**	`clinical-evidence-agent`	prompt	**Evidence gatherer.** Uses gpt-5o-mini. Tools: `azure_ai_search` (clinical-guidelines, payer-policies), FHIR functions. Returns structured evidence JSON to the Appeals Agent.
8	**Letter Agent**	`letter-agent`	prompt	**Letter writer.** Uses gpt-5o. Tools: `azure_ai_search` (letter-templates, appeal-precedents). Takes evidence + decision, drafts a formal appeal letter with citations. Output goes to Letter Reviews queue for HITL approval.
DATA & STORAGE				
9	**Azure AI Search**	`srch-appeals`	Standard S1	Hosts RAG indexes. Connect to the Foundry account as a **Foundry Connection** (type: Azure AI Search). Grant the Foundry managed identity `Search Index Data Reader` role. Engineering will create these indexes: — `clinical-guidelines` (CMS NCDs, specialty guidelines)

#	Resource	Name	SKU	Setup Notes
				— `payer-policies` (per-payer coverage criteria) — `appeal-precedents` (past successful appeals — the moat) — `letter-templates` (example letters, compliance language)
10	**Cosmos DB Account**	`cosmos-appeals`	Serverless	Hosts the **Agent Work Queue** and appeal records. Create database `appeals-db` with containers: — `work-queue` (partition key: `/status`) — denial list, letter reviews, approved, eAppeal ready — `appeals` (partition key: `/claimId`) — full appeal records with evidence + letters — `audit-log` (partition key: `/date`) — every action logged
11	**Blob Storage Container**	`appeals-docs`	Hot	Create a container in the existing storage account (or create a new one: `stappealsdocs`). Stores raw denial documents, generated letters, and uploaded evidence files.
MONITORING				
12	**Application Insights**	`appi-appeals`	PAYG	If one doesn't already exist from Phase 0, create one. Captures Function logs, agent trace events, and frontend telemetry. Connect to Foundry for OpenTelemetry export.
FRONTEND — Hosting & API Proxy				
13	**Azure Static Web App**	`swa-appeals`	Free	Hosts the React SPA frontend **and** the managed Azure Functions proxy (included free with SWA — no separate Functions App needed). The proxy forwards `/api/*` requests to Foundry, handling auth header translation and CORS. Free tier includes: 100 GB bandwidth, managed SSL, global CDN, 2 custom domains.
14	**SWA App Settings**	—	—	After creating the SWA, set these application settings (Configuration → Application settings): — `FOUNDRY_ENDPOINT` = `https://foundry-appeals.services.ai.azure.com` — `FOUNDRY_PROJECT` = `appeals-project` — `FOUNDRY_API_KEY` = (from Foundry account keys) — `APPLICATIONINSIGHTS_CONNECTION_STRING` = (from appi-appeals) These are read by the managed Functions proxy at runtime.

Foundry Connections to Create

After provisioning the resources, create these **Connections** inside the Foundry project (AI Foundry portal → Project → Settings → Connections). Agents reference connections by name — they don't need raw endpoints or keys.

Connection Name	Type	Points To	Used By
`ai-search-connection`	Azure AI Search	`srch-appeals`	All agents that use `azure_ai_search` tool — Clinical Evidence, Letter Agent

Connection Name	Type	Points To	Used By
`cosmos-connection`	Custom / API key	`cosmos-appeals`	Tool functions that read/write the work queue and appeal records
`blob-connection`	Azure Blob Storage	`appeals-docs`	Document upload/download for denial letters and evidence

Quick Reference: What Goes Where

Component	Lives In	Who Creates It
Foundry Account + Project	Azure Portal	Infra team (this doc)
Model Deployments (3)	Foundry Account	Infra team (this doc)
AI Search Service	Azure Portal	Infra team (this doc)
AI Search Indexes (4)	AI Search	Engineering (after infra provisions the service)
Cosmos DB + Containers	Azure Portal	Infra team (this doc)
Blob Container	Storage Account	Infra team (this doc)
Foundry Connections (3)	Foundry Project	Infra or Engineering
Agent Definitions (3)	Foundry Project	Engineering
Orchestrator Workflow YAML	Foundry Project	Engineering
Static Web App + Functions Proxy	Azure Portal	Infra team (this doc)
SWA App Settings (4 values)	SWA Configuration	Infra team (this doc)
Tool Azure Functions	Managed Functions in SWA / ADO Repo	Engineering
Frontend Code	SWA / ADO Repo	Engineering

Infra Provisioning Checklist

Confirm before handing off to engineering

- ☐ AI Foundry account `foundry-appeals` created with managed identity enabled
- ☐ Foundry project `appeals-project` created in the portal
- ☐ Model deployment: `gpt-5o` — status Succeeded, 80K TPM
- ☐ Model deployment: `gpt-5o-mini` — status Succeeded, 80K TPM
- ☐ Model deployment: `text-embedding-3-large` — status Succeeded, 350K TPM
- ☐ AI Search `srch-appeals` created (Standard S1), Foundry MI granted Search Index Data Reader
- ☐ Cosmos DB `cosmos-appeals` created (Serverless), database + 3 containers created
- ☐ Blob container `appeals-docs` created
- ☐ Application Insights `appi-appeals` created (or reuse existing)
- ☐ Static Web App `swa-appeals` created (Free tier) and reachable at default Azure URL

- [ ] SWA app settings configured: FOUNDRY_ENDPOINT, FOUNDRY_PROJECT, FOUNDRY_API_KEY, APPLICATIONINSIGHTS_CONNECTION_STRING
- [ ] SWA deployment token captured and provided to engineering (for ADO pipeline or manual deploy)
- [ ] Foundry Connection: `ai-search-connection` → srch-appeals
- [ ] Foundry Connection: `cosmos-connection` → cosmos-appeals
- [ ] Foundry Connection: `blob-connection` → appeals-docs
- [ ] Engineering team granted Contributor on new resources + Cognitive Services User on Foundry
- [ ] Handoff email sent with: Foundry endpoint URL, project name, AI Search endpoint, Cosmos endpoint, SWA URL, SWA deployment token

** Terraform and ADO pipelines come later. For now, manual portal provisioning gets the team unblocked fastest.*

*** Agent definitions (rows 6-8) are listed for context so infra understands what's being built. Engineering creates those after infra provisions the account and models.*

**** The SWA deployment token is a secret — share it securely. Engineering uses it to deploy the frontend build output (and the managed Functions proxy code) from their local machine or ADO pipeline.*